

业务全景与端到端主线

gomall 一次购物穿过的 15 个域——怎么连 / 怎么读 / 卡在哪

gomall · 业务侧 deck

业务全景：gomall 到底在做什么

系统架构：分层拓扑与领域地图

端到端主线：一次成功购物的”黄金路径”

贯穿全局的 4 个硬骨头

订单状态机：把 7 跳串成一条可查的线

怎么读这套 deck：路线与角色

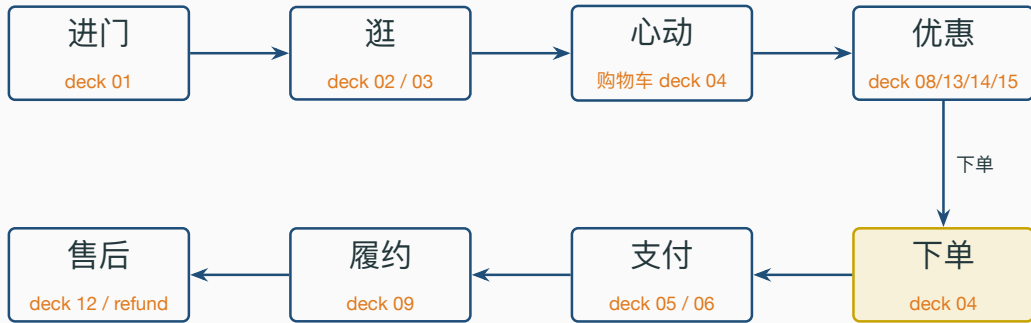
业务全景：gomall 到底在做什么

先认人：一笔订单背后站着 5 个角色

- **C 端用户**：进门 → 逛 → 下单 → 收货。只关心快、对、不丢。
- **商家 (Boss)**：订单是”我能不能发货”的唯一源头；货卖出去钱要到账。
- **运营**：办活动拉量——优惠券 / 满减 / 秒杀 / 拼团 / 预售，预算不能被刷穿。
- **客服**：用户来问”我的钱去哪了 / 货到哪了”，得有一条能查的状态线。
- **SRE**：大促洪峰别把库存卖超、别把数据库打挂，挂了要能降级。

这套 deck 按**业务域**垂直切成 15 份，每份讲透一个域。
本份是横向的”地图”：把 15 个域串成用户真实走过的一条线。

业务全景图：一次购物穿过的 8 个环节



贯穿全局：库存防超卖 (07) · Outbox 一致性 (11) · 流量治理 (10) · 优雅降级

15 份 deck 索引：域 → 在哪深讲

#	业务域	一句话
01	用户与鉴权	注册 / 登录 / JWT 双 token / RBAC
02	商品展示	list / show / Cache Aside + HTTP cache
03	商品搜索	LIKE → ES → ES+Milvus 混合召回
04	购物车 → 下单	库存预扣 + DB+outbox 同事务（主线核心）
05 / 06	支付（法币 / Web3）	余额加密 + 熔断幂等 / Escrow + 链上对账
07	库存防超卖	两桶 Lua (available / reserved) + Saga 回滚
08 / 13	营销 / 满减引擎	券 / 秒杀 / 红包 / 满减最优解
09	订单生命周期	状态机 + 延迟关单双保险
10 / 11	流量治理 / 一致性	限流熔断削峰 / Outbox + Saga
12	商家后台	merchant 三层鉴权 + 可观测性
14 / 15	拼团 / 预售	成团状态机 / 定金尾款两段付

docs/slides/README.md §deck 索引；本表为 15 份现有 deck 的横向汇总

业务承诺：我们对外兜什么底（SLO）

一个交易系统的可信度，不在于功能多，而在于敢把哪几条线写进合同——出了问题要赔的那种。gomall 把承诺分成 P0~P3 四档，资源向核心交易倾斜。

等级	典型场景	可用率	p99 延迟
P0 核心交易	下单 / 支付	99.95%	< 500ms
P1 核心读	商品详情 / 订单列表	99.9%	< 200ms
P2 营销秒杀	抢券 / 秒杀 / 红包	99%	< 200ms
P3 信息类	轮播 / 分类	99%	< 1s

- **钱货不丢**：支付与库存、资金流水落同一事务，崩了整笔回滚，绝不出现“扣了钱没货”。
- **happy path 不被周边阻断**：优惠 / 搜索 / 通知挂了，下单支付照走。
- **宁可下游慢、不能上游 429**：P0 接口（下单 / 支付）靠缓存 + 异步削峰扛峰值。

一笔支付从来不是“给买家余额减个数”。买家少的钱必须有人多——商家。复式记账让每一分钱都有去向，对账时 $\text{SUM}(\text{debit})$ 永远等于 $\text{SUM}(\text{credit})$ 。

Listing 1: 同事务内一借一贷，方向相反

```
1 // 买家扣款流水：与余额变动同事务追加
2 ledgerDao := money.NewLedgerDaoByDB(tx) // 接管调用方 tx
3 ledgerDao.AppendTransaction(uId, order.ID,
4     money.DirectionDebit, payable, buyerBalanceAfter, money.BizTypeOrderPay)
5 // 卖家入账流水：方向与买家相反，金额相同
6 ledgerDao.AppendTransaction(bossID, order.ID,
7     money.DirectionCredit, payable, bossBalanceAfter, money.BizTypeOrderPay)
```

- **同事务**：NewLedgerDaoByDB(tx) 复用余额变动那把锁的同一 tx，流水与余额原子提交。
- **防重账**：(ref_order_id, direction, biz_type) 唯一索引兜底，重放回调撞唯一键报错 → 整事务回滚。
- **外部资金**：Stripe / Web3 买家不走内部钱包，debit 对手方记在清算账户

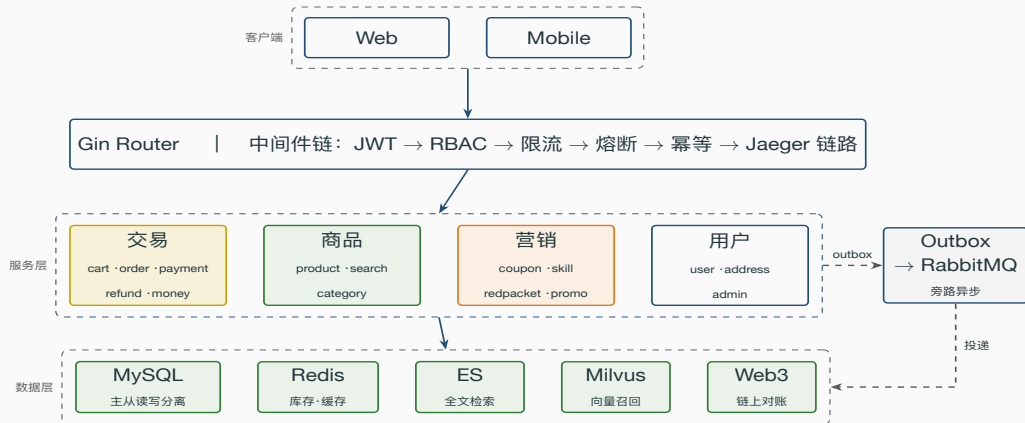
选型不是堆时髦框架，而是”这个痛点用什么扛最划算”。

组件	职责	为什么用它
Gin	HTTP 框架 / 路由 / 中间件链	裸链路 64K RPS，中间件机制干净
GORM	ORM / 事务 / 读写分离插件	dbresolver 一行接主从，事务好控
MySQL 主从	交易事务、权威数据	强一致 + 行锁，钱必须落它
Redis	库存水位、热点缓存、限流	Lua 原子扣减，零超发
RabbitMQ	异步下单、延迟关单、领域事件	TTL+DLX 做延迟队列，削峰解耦
Elasticsearch	商品全文检索	LIKE 扫全表太慢，倒排扛模糊搜
Milvus	语义向量召回	关键词搜不到的”意思相近”靠它
Web3 / go-eth	链上签名支付、nonce 防重放	钱包签名支付，listener 兜底确认

共识：交易强一致走 **MySQL**，性能与扩展走旁路（Redis / MQ / ES / Milvus），旁路挂了核心不挂。

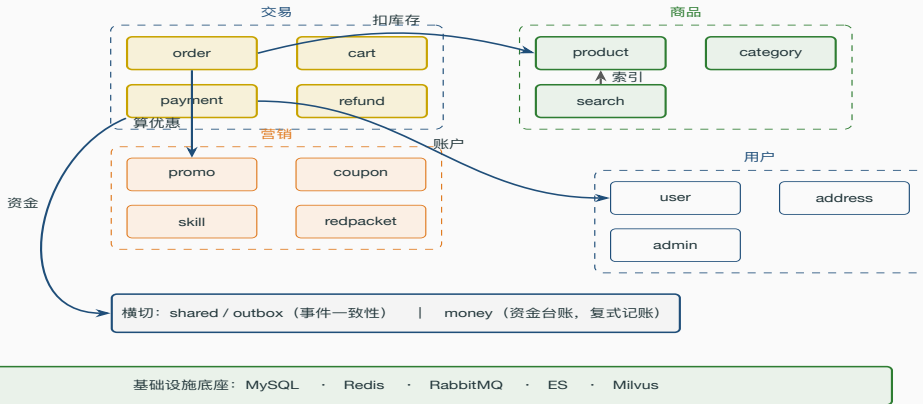
系统架构：分层拓扑与领域地图

gomall 整体架构：一条请求从客户端落到数据层



核心链路同步落库收钱，事件经 Outbox 旁路异步散给搜索 / 统计 / 履约，主线不被下游拖慢。

DDD 领域地图：22 个域按 bounded context 垂直切片



按域垂直切片，跨域只经属主领域的服务方法调用；product 与 search 已斩环（search 单向订阅 product 索引事件）。

中间件链：逐环拆解，顺序即设计

中间件的顺序不是随手排的——每一环只在前一环放行后才有意义，错一步就是漏洞。



- **JWT 鉴权** AuthMiddleware(): 解 token 拿 user_id, 挂 authed 组, 没登录直接 401 (全局: 令牌桶限流 + 链路追踪先于一切)。
- **RBAC** RequireRole("admin"): 登录之上叠角色校验, 只护 admin 组。鉴权必须在限流前——否则匿名洪水也吃配额。
- **限流 + 熔断**: 全局 TokenBucket(100,200) 防爬虫, 秒杀路由再叠滑动窗口; 支付路由专用熔断, 下游连错 5 次 open 10s。
- **幂等** Idempotency(): **放最后**——确认要执行业务时才用 Redis 锁去重, 避免给被拒请求白占幂等键。

数据层选型与边界：一个 MySQL 为什么不够

把全部活儿压给一个 MySQL，要么慢死、要么挂掉。正解按”访问形态”分流：强一致归 MySQL，高并发与检索归旁路，旁路挂了能退回 DB。

存储	扛什么	挂了怎么办
MySQL 主从	交易事务、权威读	核心依赖，不可降级
Redis	库存预扣 / 热点缓存 / 限流	缓存未命中回源 DB
Elasticsearch	商品全文检索	退回 DB LIKE 查询
Milvus	语义向量召回	退回纯 ES 关键词召回

- **读写分离**：dbresolver 写走 Sources、读走 2 个 Replicas，RandomPolicy 负载均衡。
- **搜索降级**：ES 不可用时落到 DB name/info LIKE；语义召回报错则退回纯向量——周边能力只增强体验，绝不阻断 happy path。

端到端主线：一次成功购物的”黄金路径”

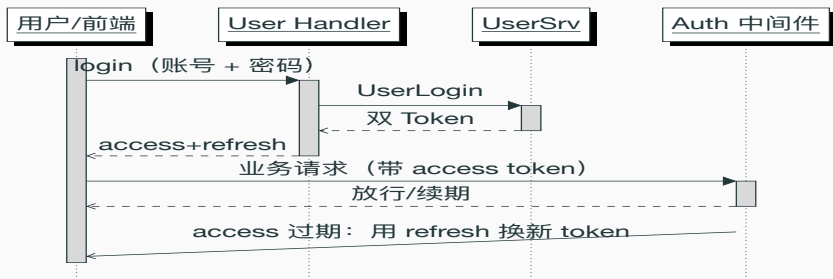
黄金路径：7 跳，每一跳都可能掉链子

1. 登录——拿到 JWT，后续每个请求带着它过鉴权中间件。
2. 看商品——详情走 Cache Aside，热点不打穿 DB。
3. 加购 / 选地址——没地址下不了单，默认地址兜底。
4. 下单——算优惠 → 预扣库存 → 一个事务落订单 + 事件。
5. 支付——扣余额 + 改状态 + 真正消库存，同事务发”已支付”事件。
6. 发货 / 收货——状态机推进，每一步只允许合法迁移。
7. 结清——订单 Completed；中途不付钱则 15 分钟自动关单。

这条线横跨 user / product / cart / promo / order / inventory / payment 七个包，
任何一跳的失败都不能让用户”钱货两失”——这是整套设计的第一性原则。

登录这一跳：密码校验 → 双 Token 签发 → 中间件续期

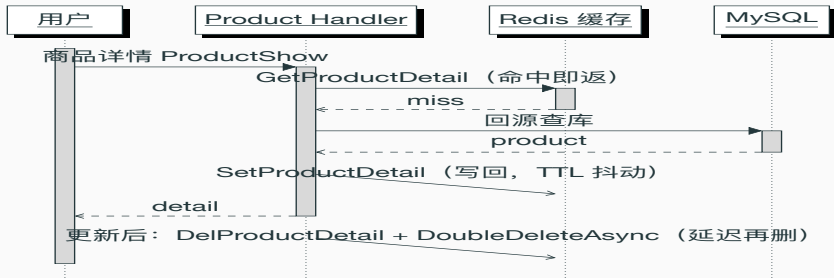
access_token 短命扛日常请求，refresh_token 长命只用来换新；前端无感续期，服务端无状态。



`internal/user/service.go:86 UserLogin (CheckPassword → GenerateToken); pkg/utils/jwt/jwt.go`
`GenerateToken/ParseRefreshToken; middleware/jwt.go:16 AuthMiddleware 续期`

看商品：Cache Aside 读路径 + 更新时延迟双删

读：先 Redis，命中即返；未命中回源 MySQL 再写缓存（空值也缓存，挡穿透）。写：先删缓存，过一会儿再删一次，盖掉并发读回填的脏数据。



`internal/product/service.go:41 ProductShow` (回源 + 写回 + 更新双删); `repository/cache/product.go`

`GetProductDetail/SetProductDetail/DoubleDeleteAsync`

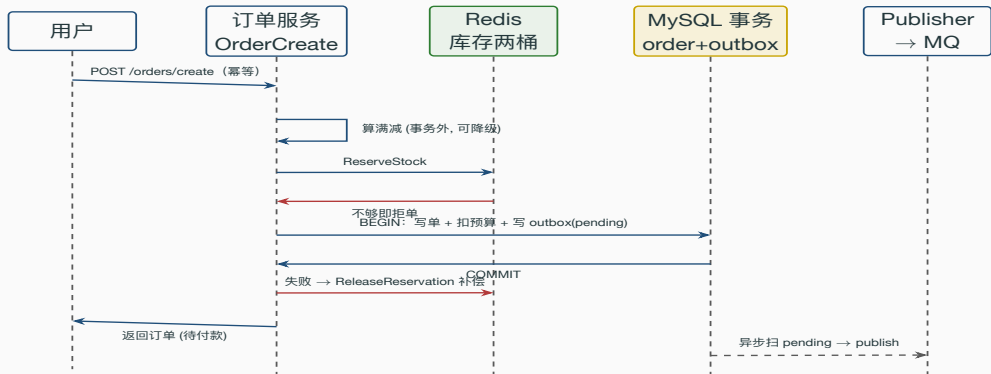
加购与地址校验：下单前的两道业务门槛

加购物车与下单，钱和卖家一律以服务端商品表为准，客户端传来的 boss_id / money 一概不信。

- **加购以商品表卖家为准**：CartCreate 先按 product_id 反查商品，拿真实 BossID 落库；购物车里的 boss_id 不信客户端，否则脏数据会把货款引到错误卖家。
- **没地址下不了单**：OrderCreate 第一步就校验收货地址归属——address_id 同样可篡改，EnsureOwned 校验该地址确属当前登录用户，否则直接拒单。
- **异步下单同样先校验**：走异步队列下单时，地址归属在入队前同步校验，失败快速拒绝，避免白白预扣库存、投递无效任务。

环节	可信来源	失败处理
加购卖家 BossID	商品表 GetProductById	商品不存在直接拒
下单单价/卖家	resolveProductPricing	定价非法拒单，不回退客户端值
收货地址	EnsureOwned（属当前用户）	不属本人立即拒单

端到端时序：下单这一跳的内部（先预扣、再事务、后通知）



internal/order/service.go:110 Reserve ·:117 Tx ·:145 outbox ·:159 释放补偿·:172 延迟关单

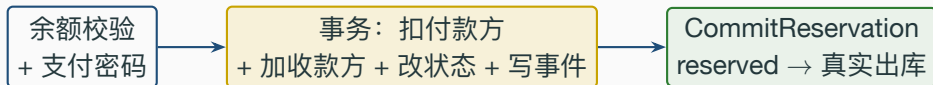
下单中枢的关键代码：顺序就是设计

Listing 2: internal/order/service.go OrderCreate 主体

```
1 // 1) 满减在事务外算：纯读路径，失败降级为无折扣，绝不阻断下单
2 promoApply, promoErr := s.promo.CalculateBestDiscount(ctx, cartItems)
3
4 // 2) Redis 预扣库存：available -> reserved，失败直接拒单，连 DB 都不碰
5 if err = cache.ReserveStock(ctx, req.ProductID, int64(req.Num)); err != nil {
6     return nil, err
7 }
8
9 // 3) 同一个事务：写订单 + 扣满减预算 + 写 outbox 事件（原子一致）
10 err = dao.NewDBClient(ctx).Transaction(func(tx *gorm.DB) error {
11     NewOrderDaoByDB(tx).CreateOrder(order)
12     s.promo.ApplyDiscountInTx(tx, order.ID, order.PromoRuleID, order.PromoDiscountCents)
13     return outbox.NewOutboxDaoByDB(tx).Insert("order", "OrderCreated", ...)
14 })
15 if err != nil { // 4) 事务失败：把第 2 步预扣的库存还回去
16     cache.ReleaseReservation(ctx, req.ProductID, int64(req.Num))
17 }
```

支付这一跳：钱和库存在这里才真正落定

- 下单时库存只是 **reserved** (预占)，钱还没动——订单状态 WaitPay。
- 支付成功才在一个事务里：扣用户余额 → 加商家余额 → 改状态 WaitShip → 写 order.paid 事件。
- 事务提交后同步 CommitReservation：把 reserved 真正消掉，库存数字才最终减少。



internal/payment/service.go:36 PayDown; repository/cache/inventory.go:98 CommitReservation

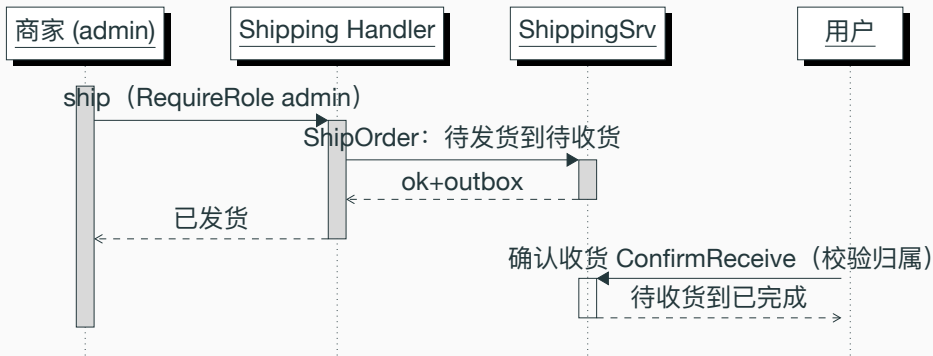
支付事务体：钱、货、状态、事件一把梭

Listing 3: internal/payment/service.go PayDown 事务体（节选）

```
1 err = orderpkg.NewOrderDao(ctx).Transaction(func(tx *gorm.DB) error {
2     order, _ := orderpkg.NewOrderDaoByDB(tx).GetOrderById(req.OrderId, uId)
3     if order.Type != consts.OrderWaitPay { // 幂等：非待付款拒绝重复支付
4         return errors.New("订单状态非未支付")
5     }
6     // 扣买家余额 (AES 密文解密校验支付密码) + 加商家余额
7     userDao.UpdateUserById(uId, buyer) // 买家 - totalMoney
8     userDao.UpdateUserById(bossID, boss) // 商家 + totalMoney
9     order.Type = consts.OrderWaitShip // 状态推进：待付款 -> 待发货
10    orderpkg.NewOrderDaoByDB(tx).UpdateOrderById(req.OrderId, uId, order)
11    return outbox.NewOutboxDaoByDB(tx).Insert( // order.paid 事件同事务落库
12        "order", "OrderPaid", "order.paid", order.ID, events.OrderPaid{...})
13 })
14 // 事务提交后：把 Redis reserved 桶真正消掉，库存数字最终减少
15 cache.CommitReservation(ctx, paidProductID, int64(paidNum))
```

履约这一跳：发货 → 收货，每步都过状态机

发货是 admin 权限的运营动作，收货是用户本人动作；两步都只认状态机里的合法迁移，且状态推进与 outbox 事件同事务，杜绝“已发货但下游收不到事件”。



满减最优解：多规则叠加时帮用户算到最便宜

用户身上可能同时命中好几条满减规则，系统帮他挑出”减得最多”的那一条；算折扣失败就降级为无折扣，绝不阻断下单。

- **先算后下单，两段分工：**结算/购物车实时展示走 `CalculateBestDiscount`（纯只读、不落库）；真正下单时在事务内 `ApplyDiscountInTx` 扣减规则预算并落账。
- **怎么算”最优”：**`PickBestPromoRule` 遍历所有命中规则，逐条算可减金额，取最大的一条；没有命中或减额 ≤ 0 就返回原价。
- **失败降级：**`CalculateBestDiscount` 报错时记 warn 日志、回退无折扣继续下单；规则畸形导致折扣 $>$ 小计时，兜底把实付夹到 0，不让用户实付为负。

两段分工一句话：**结算页只读算最优（`CalculateBestDiscount`，不落库），下单事务内才落账（`ApplyDiscountInTx`，超支回滚）；任一异常都降级为无折扣、绝不阻断成交。**

贯穿全局的 4 个硬骨头

硬骨头 1：库存防超卖——为什么不用 UPDATE WHERE stock>0

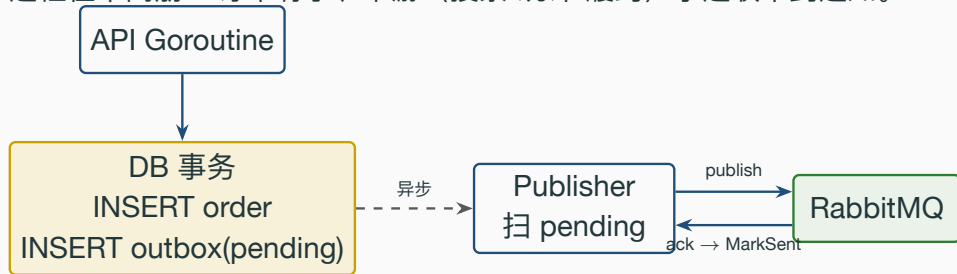
- 高并发下行锁排队 + 尾延迟爆炸；下单链路最怕在事务里等锁。
- gomall 把库存搬到 Redis，用 **两个桶**：available（可卖）/ reserved（已占）。
- 预扣 / 提交 / 释放都是 **单个 Lua 脚本原子完成**，一次 RTT，天然不超卖。

Listing 4: reserveScript: available 与 reserved 两个 key 原子流转

```
1  -- KEYS[1]=available 桶 KEYS[2]=reserved 桶 ARGV[1]=n
2  local avail = redis.call('GET', KEYS[1])
3  if avail == false then return -2 end -- 库存未初始化
4  if tonumber(avail) < tonumber(ARGV[1]) then
5      return -1 -- 不够，拒单
6  end
7  redis.call('DECRBY', KEYS[1], ARGV[1]) -- available -= n
8  redis.call('INCRBY', KEYS[2], ARGV[1]) -- reserved += n
9  return 1
```

硬骨头 2: Outbox ——解决” 订单存了但消息丢了”

双写问题：先 INSERT order 再 publish MQ，两个动作不在一个事务里。
进程在中间崩 = 订单有了、下游（搜索/统计/履约）永远收不到通知。



订单与事件**同生共死**写进一个事务；投递交给独立 publisher 重试。
至少一次投递 + 下游幂等消费 = 事件不丢。

硬骨头 3：优雅降级——核心稳如磐石，边缘随时可弃

能力	挂了会怎样	gomall 的选择
满减计算	算不出折扣	降级为 无折扣 ，照样下单
满减预算	预算抢光	事务内 改写为无折扣 ，不报错
RabbitMQ	延迟关单不可用	Redis ZSet 定时扫描 兜底关单
ES / Milvus	搜索退化	退回 DB LIKE 路径

判断标准只有一句：它是不是“用户付钱拿货”的必经环节？
是 → 进事务、保原子；不是 → 失败降级，绝不阻断 happy path。

internal/order/service.go:78 满减降级 ·:126 预算耗尽降级；cmd/main.go:69-102 ES/Milvus 静默跳过

硬骨头 4：流量治理——大促洪峰怎么扛

- **限流**：每 IP 令牌桶 100 RPS / 突发 200，挡爬虫与脚本。
- **幂等**：下单走 `middleware.Idempotency()`，用户狂点也只成一单。
- **削峰**：`/orders/enqueue` 投 MQ，前端拿 ticket 轮询，把洪峰摊平。

压测实证：幂等中间件下 **755K 次重放请求** → **恰好 1 笔订单**，首单成功后续请求由 Lua 读 done 态返回，**50K RPS / p95 2.33ms**。

`routes/router.go:41` 令牌桶；`internal/order/routes.go` 幂等 +enqueue；`stressTest/REPORT.md` § 幂等中间件

业务痛点：用户手抖狂点”提交订单”、客户端超时自动重试、弱网下 App 补发请求——同一笔下单不能变成三笔。

请求态状态机 (Redis Hash)

- init: 已发券、未执行
- processing: 首单执行中, 后到一律挡回
- done: 已完成, 缓存响应直接回放

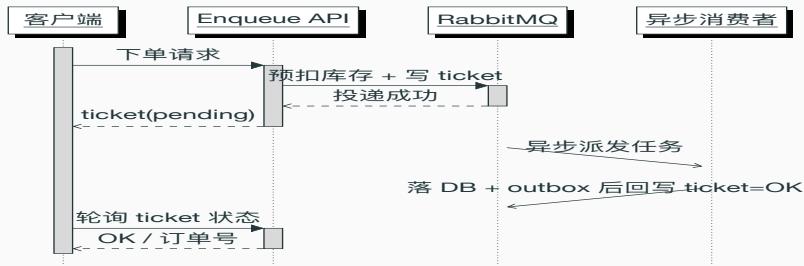
一段 Lua 单次 RTT 原子完成”读态 + 抢锁”。

Listing 5: acquire: init 到 processing 原子推进

```
1 local v = redis.call('HGET', KEYS[1], 'state')
2 if v == false then return {0, ''} end
3 if v == 'done' then return {2,
4     redis.call('HGET', KEYS[1], 'result')} end
5 if v == 'processing' then return {3, ''} end
6 -- init: 抢到执行权
7 redis.call('HSET', KEYS[1], 'state', 'processing')
8 redis.call('EXPIRE', KEYS[1], tonumber(ARGV[1]))
9 return {1, ''}
```

削峰：下单先拿 ticket，洪峰摊到 MQ

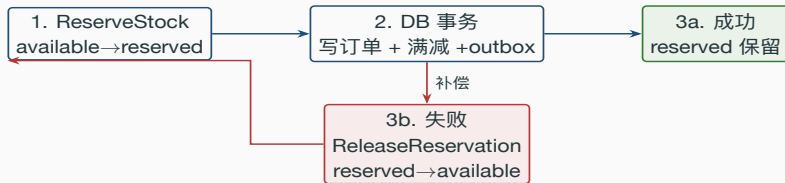
业务背景：大促零点瞬时洪峰，若每个请求都同步写库，DB 连接池秒满。改成“先预扣库存 + 投 MQ 拿 ticket 秒回”，后台慢慢消费落单，把尖峰摊平成平稳的消费速率。



ticket = 雪花 ID 存 Redis 带 TTL；写 ticket / marshal / publish 三个失败点任一出错都先 ReleaseReservation 退回预扣库存，绝不让库存空占。

Saga 库存补偿：预扣在事务外，失败必须还回

Redis 预扣库存 (available→reserved) 发生在 DB 事务之外——高并发下不让 Redis 扣减跟着行锁排队。代价：后续失败必须由代码主动补偿还回，否则库存被”幽灵占用”。



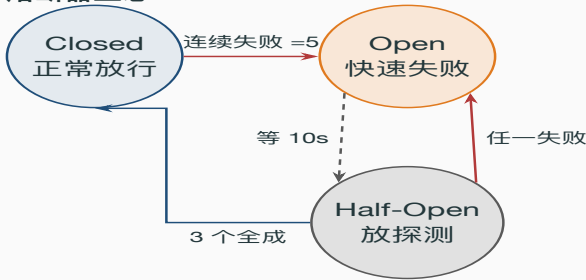
- 两个桶：available (可下单) / reserved (已占未付)，支付成功才 CommitReservation 真正消耗。
- 补偿点遍布全链路：同步事务失败、异步消费失败、ticket/publish 失败——每处都调 ReleaseReservation。
- 对账兜底：ScanReservedProductIDs 用 SCAN 增量枚举 reserved 桶，定时比对 DB

两道闸门：令牌桶在入口按 IP 限速，挡住脚本刷接口；熔断器在出口监控下游，下游连续报错就快速失败，不让请求堆在垂死的依赖上拖垮自己。

令牌桶限流（每 IP）

- 全局 TokenBucket(100, 200)：每 IP 每秒补 100 个令牌，桶容量 200 (允许突发)。
- 取不到令牌 → 直接 Abort，返回”请求过于频繁”。
- 后台 janitor 周期回收 10 分钟无访问的 IP 条目，map 有界，防海量 IP 撑爆内存。

熔断器三态



细节：半开探测带”代际”令牌，上一轮 Open 期的迟到回报会因代际不符被

可观测性：一次下单串成一条 trace

下单跨了网关、订单、库存、MQ、DB 多个环节。出问题时光看单机日志拼不出全貌——链路追踪给每次请求发一个 `traceId`，沿调用一路透传，把散落的 `span` 串成一条完整时间线。

链路怎么串起来

- `cmd/main.go` 引入 SkyWalking agent，自动织入出入站调用埋点。
- `track` 中间件读请求头 `uber-trace-id` (Jaeger 透传格式) 续上父 `span`；缺失或解析失败则降级开本地 `span`，绝不吞掉整条请求。
- `span` 放进 `context` 往下传，DB/Redis/MQ 调用挂成子 `span`。

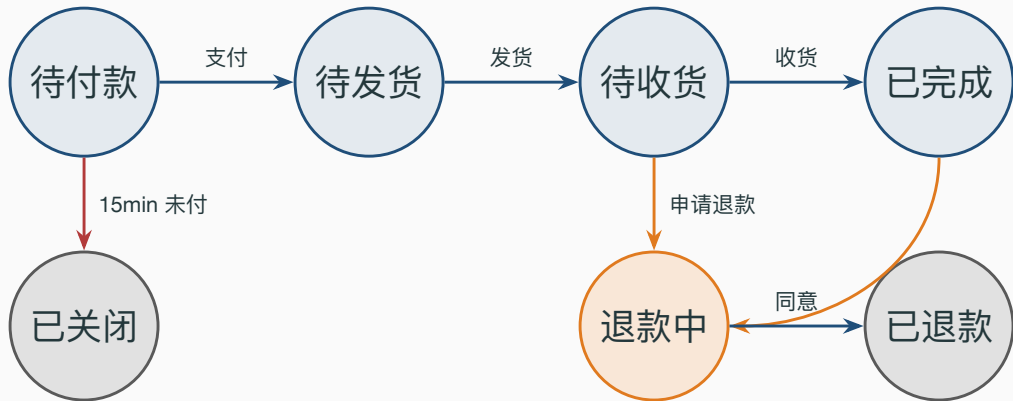
关键指标看板

指标	用途
QPS	吞吐 / 容量水位
p95 延迟	尾延迟，体感真相
错误率	非 2xx 占比

业务态（限流/熔断/幂等命中）HTTP 仍 200，单独用 `status` 码区分，避免污染错误率口径。

订单状态机：把 7 跳串成一条可查的
线

订单不是一个布尔值，是一台状态机



每一次状态变更都走 CanTransit 校验，非法迁移直接报错——
“已完成”的订单不可能跳回“待付款”，客服查到的状态永远自治。

谁来推状态：用户 / 商家 / 系统三方驱动

迁移	触发方	入口	权限
待付款 → 待发货	用户	支付成功	登录
待发货 → 待收货	商家	ShipOrder	admin RBAC
待收货 → 已完成	用户	ConfirmReceive	登录
待付款 → 已关闭	系统	延迟关单	MQ / Cron
* → 退款中	用户/客服	refund	登录

”系统”这一行最容易被忽略：没人点按钮，靠 RabbitMQ 延迟消息 + DB Cron 定时扫描**双保险**把僵尸订单关掉、库存还回。

internal/order/shipping.go:36 ShipOrder ·:73 ConfirmReceive; internal/order/cancel.go:19 CancelUnpaidOrder

延迟关单：MQ 延迟队列 + DB Cron 双保险

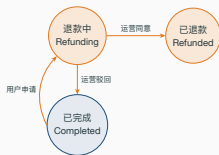
业务承诺：下单后 15 分钟不付款，自动关单并把库存预占退回 available，绝不让“占着不买”的单子把库存占死。



- **主路：**延迟队列 TTL 到期 → 死信交换机 → 消费者调 CancelUnpaidOrder。兜底：MQ 挂了也无妨，DB Cron 每 5 分钟扫 WaitPay 超时单补偿关单。
- 两路共用同一幂等关单 CloseOrderWithCheck：只对未付款生效，二次调用直接 no-op。

退款子状态机：状态推进与资金/库存回退分离

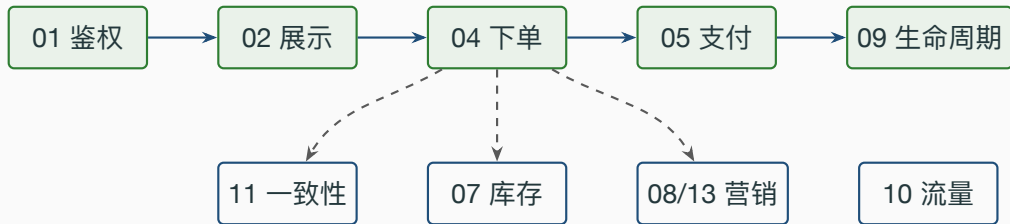
退款拆两步走：先在订单事务里推进状态机 + 写 outbox 事件，真正的”还库存 + 退钱”由事件消费者 SettleRefund 单事务原子落地。



- **进入**：仅 WaitShip / WaitReceive / Completed 可发起退款（未付款走关单，不走退款）。
- **终态落地**（SettleRefund 单事务原子）：买家 credit / 卖家 debit（复式记账）+ RollbackStock 加回库存；退款额取实付口径 FinalCents，不重复退满减优惠。
- **幂等**：台账 (order_id, credit, refund) 唯一索引 + 入账前存在性预检，重复投递只退一次。

怎么读这套 deck：路线与角色

阅读路线图：主干一条线，支线随用随查



(下单为枢纽，需要深究时下钻到支线)

第一遍：沿绿色主干 01→02→04→05→09 走一遍，建立全局。

第二遍：哪块想深究，从 04 下钻到 07 / 11 / 08 / 10。

各 deck 末页” 配套 deck” 指向相邻域

按角色取你最关心的几份

- 后端新人：01 → 02 → 04，把”一个请求怎么从路由走到 DB”吃透。
- 做交易的：04 下单 + 05/06 支付 + 09 生命周期，这是钱货闭环。
- 做营销的：08 营销 + 13 满减引擎 + 14 拼团 + 15 预售，看预算与防刷。
- **SRE / 架构**：07 库存 + 10 流量 + 11 一致性，三份是系统抗压的底座。
- 面试备战：每份末尾的 Q&A 两页 + 本份”硬骨头 4 连”。

不必从头读到尾。先用本份建立地图，再按你的角色挑 3~4 份深读，读完任意一份都能回到这张全景图定位自己在哪。

docs/slides/ 共 15 份 + 本总览；*docs/blog/* 同主题深度补充

这套架构的一句话总结

核心链路（扣库存 + 落单 + 收钱）必须稳如磐石，周边能力（优惠 / 通知 / 搜索 / 关单）全部可降级。

顺序即设计：

- 能在事务外做的绝不进事务——满减、库存预扣。
- 进了事务就保证原子——订单 + 优惠预算 + outbox 事件。
- 事务外的副作用必须能补偿——库存释放、延迟关单。

internal/order/service.go:63-178 OrderCreate —函数浓缩全部原则

面试 Q&A (1/2)

Q1: 为什么库存预扣放 Redis 而不放 DB 事务里?

A: DB 行锁在高并发下排队、尾延迟爆炸; Redis 两桶 Lua 一次 RTT 原子完成, 下单链路不被锁拖住。失败由 ReleaseReservation 补偿 + 对账兜底。

Q2: 下单为什么要 outbox, 不直接发 MQ?

A: 直接发会有双写问题——订单落库与消息投递不在一个事务, 中间崩溃就丢事件。outbox 把事件写进同一个 DB 事务, 再由 publisher 至少一次投递。

Q3: 满减算挂了为什么不直接报错?

A: 优惠是锦上添花, 不是下单前提。SLO 承诺保护 happy path, 所以算失败、预算耗尽都降级为无折扣, 用户照样能成交。

Q4: 事务提交后才发 MQ 延迟关单，万一 MQ 挂了订单永远不关？

A: 双保险。RabbitMQ 延迟队列 (TTL+DLX) 为主，DB Cron (每 5min 扫 WaitPay 超时单) 兜底，MQ 不可用也能关单释放库存；两路共用幂等关单，重复触发只生效一次。

Q5: 支付成功后库存是怎么真正减少的？

A: 下单只 reserved (预占)；支付事务改完状态后同步 CommitReservation，把 reserved 真正消掉。未支付被关单则 ReleaseReservation 还回 available。

Q6: 游标分页相比深分页到底强多少？

A: 压测里订单列表游标 + 缓存 58,406 RPS / p95 5ms，旧版深分页 8.3 RPS / p95 15.95s ——吞吐约 7,000×、p95 约 3,200×。

压测数据汇总（k6，本地单机，order 表 ~6M 行）

场景	口径	RPS	p50	p95
幂等下单	同 Key 重放	50,319	0.50ms	2.33ms
订单列表（游标 + 缓存）	新版	58,406	1.04ms	5.0ms
订单列表（旧版深分页）	OFFSET 2M	8.3	10.5s	15.95s
商品详情（DB+PK）	无缓存	62,226	0.73ms	3.01ms
Ping 基线	裸 gin	64,254	0.91ms	3.51ms

- 幂等中间件：50 VU 同一 Idempotency-Key 持续打 /orders/create，15s 累计 **755,033** 次请求 → **DB 实际只落 1 笔订单**。首单后 Lua 读 done 态缓存返回，成本等同读 Redis 单值。
- 游标分页 vs 深分页：吞吐 ~**7,000**×、p95 ~**3,200**×。旧版每次 COUNT(*)+OFFSET 2M 扫全表；新版 WHERE id<last_id LIMIT 20 走 PK 倒序。

面试 Q&A (3/3)：一致性·熔断·检索

Q7：MySQL 主从读写分离下，读一致性怎么保证？

A：普通读走从库分摊负载；对一致性敏感的资金/库存路径全部在事务内 FOR UPDATE (LockTwoUsersForUpdate / GetUserByIdForUpdate)，事务与行锁读由 dbresolver 强制路由到主库，绕开从库延迟。

Q8：熔断器半开态怎么探测下游恢复？

A：Open 超过 OpenTimeout（默认 10s）转半开，只放 HalfOpenMaxReq（默认 3）个探测；连续成功达阈值回 Closed，任一失败立刻回 Open。代际（generation）机制丢弃上一轮迟到回报，不误触发刚恢复的熔断器。

Q9：已有 ES，为什么还要 Milvus 向量召回？

A：ES 擅长关键词精确匹配，”显瘦的连衣裙”这类语义查询命中差；Milvus 走 embedding 语义召回，双路召回后加权融合（min-max 归一）。ES 挂了降级走纯向量，不让语义检索整体不可用。

面试 Q&A (4/4): 投递去重·ID 防撞·资金对账

Q10: outbox 至少一次投递, 下游怎么防重复消费?

A: 不追求”恰好一次”, 而是至少一次投递 + 消费端幂等吸收重复。落地手段是业务台账唯一索引——满减退预算 `promo_release(order_id)`、退款台账 (`ref_order_id`, `direction`, `biz_type`), 重复投递撞键即幂等返回。

Q11: 雪花 ID 多副本部署怎么防 ID 撞车?

A: 每副本必须配 `SNOWFLAKE_NODE_ID` (或 `NODE_ID`), 范围 `[0,1023]`; 启动强校验——未配置或越界直接 Fatal 拒绝启动, 仅 `SNOWFLAKE_ALLOW_DEFAULT=true` 时退回 node 0 (限本地开发)。

Q12: 支付为什么要复式记账? 解决什么对账问题?

A: 余额密文落库不可对账; 每次变动追加一条不可变明文流水, 一次转移成对出现 (买家 debit / 卖家 credit), 恒等式 $\text{SUM}(\text{debit}) = \text{SUM}(\text{credit})$ 。外部入口 (Stripe/Web3) 对手方记清算账户 (`user_id=0`), 守恒不破, 可逐笔追溯资金来龙去脉。

代码位置一览

登录 / 鉴权 *internal/user/service.go:82 UserLogin; pkg/utils/jwt/jwt.go:25 GenerateToken;
middleware/jwt.go:16 AuthMiddleware*

商品 / 购物车 *internal/product/service.go:40 ProductShow; internal/cart/service.go:29 CartCreate*

下单中枢 *internal/order/service.go:63–178 OrderCreate (满减:110 Reserve · 117 Tx · 145 outbox · 159 补
偿 · 172 延迟关单)*

库存两桶 *repository/cache/inventory.go:78 Reserve · 98 Commit · 113 Release*

满减引擎 *internal/promo/service.go:162 CalculateBestDiscount · 234 ApplyDiscountInTx*

Outbox *internal/shared/outbox/repo.go:26 Insert · 44 FetchBatch · 53 MarkSent;
initialize/outbox.go:15 InitOutboxPublisher*

支付 / 履约 *internal/payment/service.go:36 PayDown; internal/order/shipping.go:36 ShipOrder · 73
ConfirmReceive*

状态机 / 关单 *internal/order/state.go:21 orderStateTransitions; internal/order/cancel.go:19
CancelUnpaidOrder*